

1) Token definition

A token is a digital unit of value recorded on a blockchain, but unlike a coin, it does not have its own blockchain. Instead, it exists on top of another blockchain and is usually controlled by a smart contract. A token can represent money, rights, ownership, access, or many other things, depending on how it is programmed.

2) Coin vs token

A coin is the native asset of its own blockchain, such as Bitcoin on the Bitcoin blockchain or Ether on Ethereum, and it is usually used to pay network transaction fees. A token, on the other hand, is built on top of an existing blockchain and uses that blockchain's security and infrastructure. So the main difference is that a coin belongs to its own blockchain, while a token depends on another blockchain.

3) What tokens can represent

Tokens can represent many types of value or rights, not just money. They may represent stablecoins, voting rights, access rights, ownership shares, loyalty points, or real-world assets like gold or property. In simple words, a token is a programmable ledger entry whose meaning depends on what the smart contract says it represents.

4) How tokens work through smart contracts

On blockchains like Ethereum, a smart contract defines the token's rules, such as balances, transfers, minting, and burning. The blockchain ledger stores who owns how many tokens, and a transfer simply updates that stored state. So a token works because a smart contract manages the rules while the blockchain records the balances and changes.

5) Stablecoin issuance and redemption

Stablecoins such as USDC or USDT are usually issued when a user sends fiat money like USD to the issuer, and in return the issuer mints the same value in tokens. When the user later

returns the stablecoin, the issuer burns the token and gives back the fiat value. This issuance-and-redemption process helps keep the stablecoin linked to the value of one real dollar.

6) Arbitrage keeping stablecoin near \$1

Arbitrage is the process that helps keep a stablecoin's market price close to \$1. If the token trades above \$1, traders can buy from the issuer at \$1 and sell in the market for profit, which pushes the price down. If it trades below \$1, traders can buy cheaply in the market and redeem for \$1, which pushes the price back up. So price differences create profit opportunities that naturally pull the stablecoin back toward its peg.

7) ART vs EMT

An Asset-Referenced Token (ART) is a token whose value is linked to multiple assets, such as a basket of fiat, commodities, or crypto. An Electronic Money Token (EMT) is tied to a single fiat currency like USD or EUR and is more directly payment-oriented. So the simplest difference is: ART refers to multiple assets, while EMT is pegged to one specific fiat currency.

8) Financial transaction elements

A proper financial transaction must include the payer, payee, amount, and authorization. The payer tells who is sending the value, the payee tells who receives it, the amount tells how much is being transferred, and the authorization proves that the payer actually approved the transaction. Without these four elements, a transaction would be incomplete or insecure.

9) Account-based model

In the account-based model, the system keeps a list of accounts and their balances, and a transaction simply subtracts from the sender and adds to the receiver. A transaction is valid if the sender has enough balance to cover the amount. This model feels similar to normal banking, and Ethereum is an example of a blockchain that uses this account-based approach.

10) Replay attack

A replay attack happens when a valid old transaction is copied and submitted again, causing it to execute more than once. For example, if Alia signs a transaction sending money to Babar, an attacker might replay the same signed transaction again and drain her balance twice. So replay attacks are dangerous because a valid message should not remain forever reusable.

11) SEQ / unique identifier to stop replay

Replay attacks can be prevented by adding a unique sequence number or identifier to each transaction before signing it. This makes every transaction distinct, even if payer, payee, and amount are the same as before. The system then checks whether a sequence number has already been used, and if it appears again, the replayed transaction is rejected.

12) UTXO model

The UTXO model, used by Bitcoin, does not mainly track balances as accounts. Instead, it tracks spendable outputs from previous transactions, called Unspent Transaction Outputs. A Bitcoin transaction consumes one or more old UTXOs and creates new UTXOs, so value moves by destroying old spendable outputs and creating fresh ones.

13) Bitcoin legacy transaction format

A Bitcoin legacy transaction includes inputs, outputs, and some extra fields such as lock_time. Inputs point to earlier unspent outputs and provide unlocking data, while outputs create new spendable outputs with values and locking conditions. So the legacy format is basically a structured way of saying: "I want to spend these old outputs and create these new ones."

14) Input vs output

An input in Bitcoin points to an earlier unspent output and tries to spend it by providing valid unlocking data. An output, on the other hand, is a new piece of value created by the transaction and locked to a future owner. So inputs consume old value, while outputs create new spendable value.

15) Previous transaction ID and index

Each Bitcoin input must say exactly which earlier output it wants to spend. It does this using the previous transaction ID, which identifies the earlier transaction, and the index, which tells which output inside that earlier transaction is being referenced. This is necessary because one transaction may have multiple outputs, so the index selects the correct one.

16) ScriptSig

ScriptSig is the unlocking script provided by the spender in a Bitcoin transaction input. In the common legacy form, it usually contains the spender's signature and public key. Its job is to provide evidence that the person trying to spend the old output is actually authorized to do so.

17) ScriptPubKey

ScriptPubKey is the locking script stored in the previous output being spent. It contains the rules that must be satisfied for that output to be unlocked, usually in the form of Bitcoin Script instructions. In simple words, ScriptPubKey says, "Here is what someone must prove in order to spend this coin."

18) P2PKH

P2PKH stands for Pay to Public Key Hash, which is a common Bitcoin legacy transaction type. In this format, the output is locked to the hash of the recipient's public key rather than to the full public key itself, because the hash is shorter and more efficient to store. To spend that output later, the rightful owner must provide the correct public key and a valid signature.

19) Stack-based execution

Bitcoin Script is executed using a stack, which is a last-in-first-out data structure. The spender's ScriptSig first pushes unlocking data like the signature and public key onto the stack, and then

ScriptPubKey runs instructions that test whether that data is valid. If the script ends with a valid true result, the spend is accepted.

20) OP_DUP, OP_HASH160, OP_EQUALVERIFY, OP_CHECKSIG

These are the core instructions in a common P2PKH locking script. OP_DUP duplicates the public key, OP_HASH160 hashes a copy of it, OP_EQUALVERIFY checks that the computed hash matches the stored expected hash, and OP_CHECKSIG verifies that the provided signature is valid under the provided public key. Together, these operations prove that the spender owns the correct key and authorized the transaction.

21) Change output

In Bitcoin, a spender usually consumes a full previous UTXO even if they only want to pay part of it. The remaining value is not lost; instead, it is sent back to the spender as a new output called the change output. So if Alia spends a larger old UTXO to pay Babar, one new output may go to Babar and another may return the leftover amount back to Alia.

22) Final “True” meaning valid spend

At the end of Bitcoin Script execution, the final result must be True for the spend to be valid. This means the provided public key matched the expected public key hash and the signature check succeeded. So when the script ends in True, the network accepts that the spender had the right to unlock the old output and create the new outputs.